

Обработка на грешки и изключения в C++

Семинар 12
Серхан

Изключение - обект, хвърлен при необичайна ситуация, прекъсва нормалния поток на изпълнение, прераба контрола нагоре по call stack-а

поехсерт - гаранция, че функцията никога не хвърля изключение

std::optional<T> - обвивка за стойност, която може да липсва

std::expected<T, E> - обвивка за стойност или грешка

Грешките в програмата са два вида:

* отаквани ситуации - търсим нещо и не го намираме; въведен е невалиден формат

* изключителни ситуации - конструктора не може да създаде валиден обект; файл не се отваря; паметта свършва

throw е скъпа операция, времето е непредсказуемо.

① enum class Error Codes - четимо и типова безопасно, но кодът за грешка може да се игнорира

② Изключения - throw, try, catch

• throw - хвърля обект, а не тип. Незабавно прекъсва функцията и кода надолу не се изпълнява.

throw 42;

throw "грешка" ← лоши практики

throw std::runtime_error("Описание"); ✓

try и catch - хващане

try {

... ← ① тук се хваща грешката

}
catch (const std::invalid_argument & e) ② влизаме в блока catch
{ cout << "Хваната грешка: " << e.what(); }

... ④ продължаваме след catch блока

③ принтираме

- При няколко catch блока:
 - проверяват се отгоре надолу
 - изпълнява се първия съвпадение
 - от най-специфичен към най-общ

catch (const std::exception & e) → улавя всички
{ // логика } std::exception

catch (...) ← улавя абсолютно всичко
(int, char* и т.н.)

При хвърляне на изключение:

1. Текущата функция прекъсва
2. Стека се размотава
3. Локалните обекти се унищожават (деструктори в обратен ред)
4. Търси се първия catch, който улавя изключението

Хващане по стойност & по референция:

catch (std::exception e) — по стойност — лоша практика
→ обектът се нарязва, полиморфизмът се губи

catch (const std::exception & e) — по const референция ✓
→ избягва slicing, не прави копие, позволява полиморфизъм

Иерархия:

`std::exception` (базов - метод `what()`)

└ `std::logic_error` (грешки в логиката - предотвратими) ^{- грешка на програмиста}

└ `std::invalid_argument` - невалиден аргумент

└ `std::out_of_range` - индекс извън граница

└ `std::length_error` - невалидна дължина

└ `std::domain_error` - невалиден домейн

└ `std::runtime_error` (грешки при изпълнение - непредотврат.) ^{- зависи от средата, не от кода}

└ `std::overflow_error` - аритметично препълване

└ `std::underflow_error` - аритм. подпълване

└ `std::range_error` - резултатът е извън обхвата

Метод `what()` - връща текстово описание на грешката

└ виртуален метод в `std::exception`

`const char* what() const noexcept;`

- при собствен клас се предефинира с `override`

Собствен клас за изключение - наследява `std::exception`
- предефинира метода `what()`

• `throw;` - повторно хвърляне без загуба на тип
(оригиналният обект с оригинален тип)

`std::current_exception` - запазва изключението за по-късно и го хвърля отново

Изключения в конструктора: ако обектът не може да бъде валидно конструиран → грешка

- При хвърлено изключение:
 1. Обектът не е създаден
 2. Деструкторът му не се вижда
 3. Деструкторите на създадените обекти се виждат автоматично

• В деструктор — без хвърляне на грешки
↓
те са поехсерт

• Ако в деструктор се хвърли грешка, стекът се размотава и се викат деструкторите на създадените обекти. Ако деструкторът хвърли ново изключение $\Rightarrow$ 2 изключения и се вика `std::terminate`
Решението: в деструктора хващай (блока `try` и `catch`)

`std::optional<T>` - представя стойност, която може да липсва
- заменя `nullptr` проверки, магически стойности

⊕ при функция, която може да не намери резултат

⊕ `optional` полета в клас

⊖ колекции

⊖ `ownership` $\rightarrow$ използвай `unique_ptr`

`std::expected<T, E>` - съдържа или валидна стойност от тип `T` или грешка от тип `E`
- носи информация защо няма стойност